

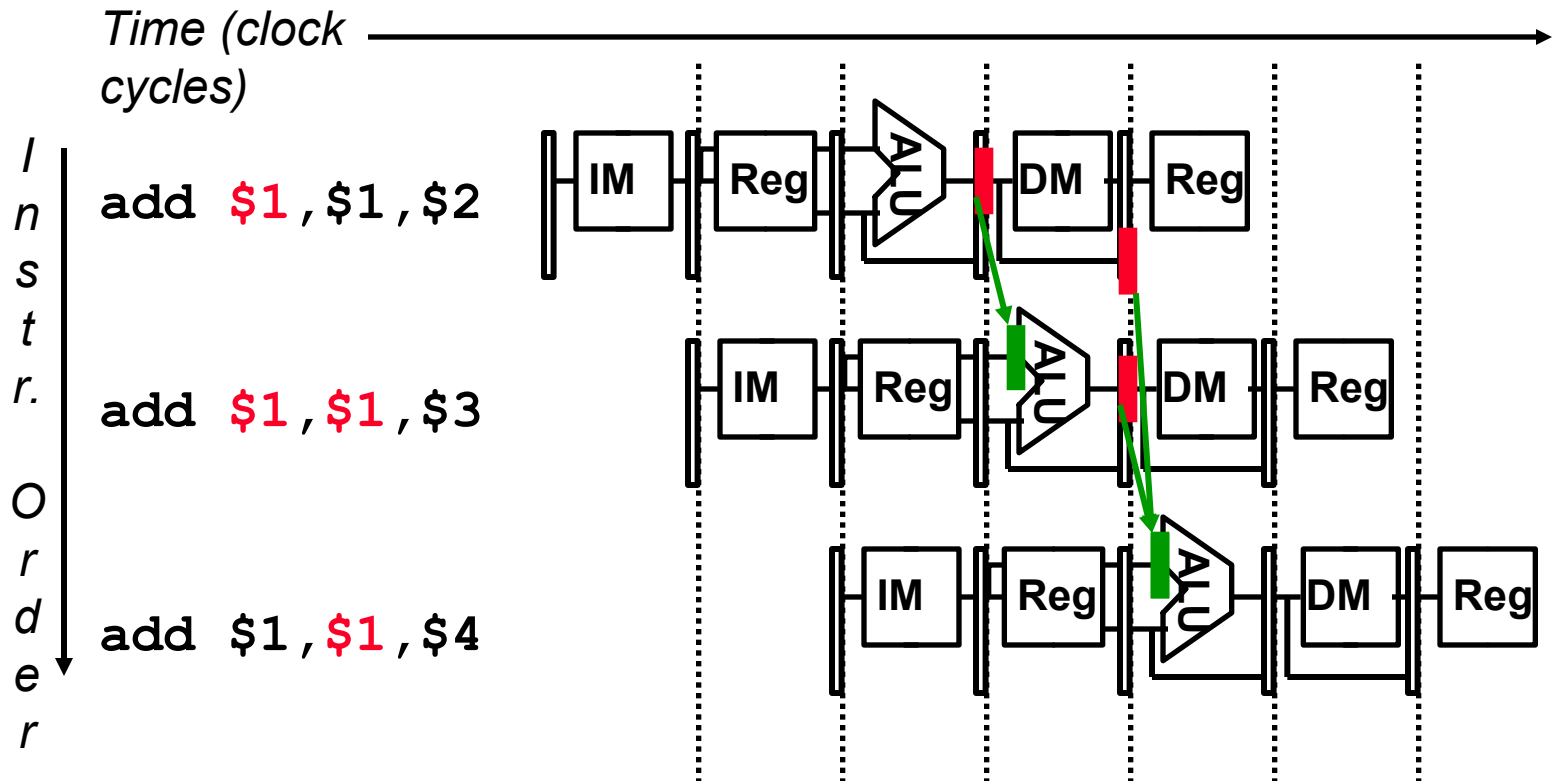
Review



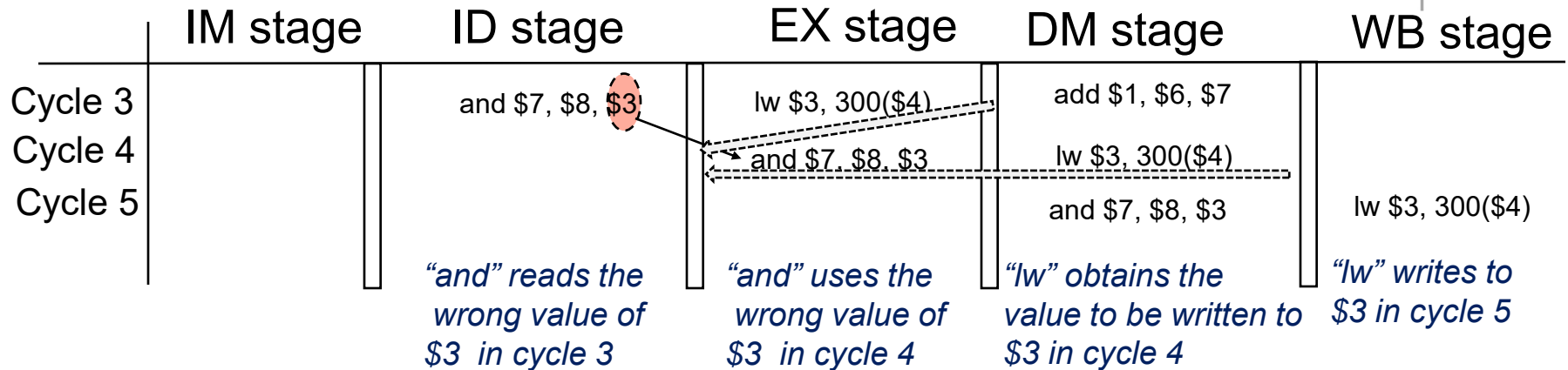
- Last Class:
 - Review of pipeline
 - Single cycle processor → CCT problem
 - Pipelined execution
 - Structural and data hazard
 - Data forwarding
- Today's class:
 - Control hazard
 - Multi-cycle pipelines

Yet Another Complication!

- Another potential data hazard can occur when there is a conflict between the result of the WB stage instruction and the MEM stage instruction – which should be forwarded?

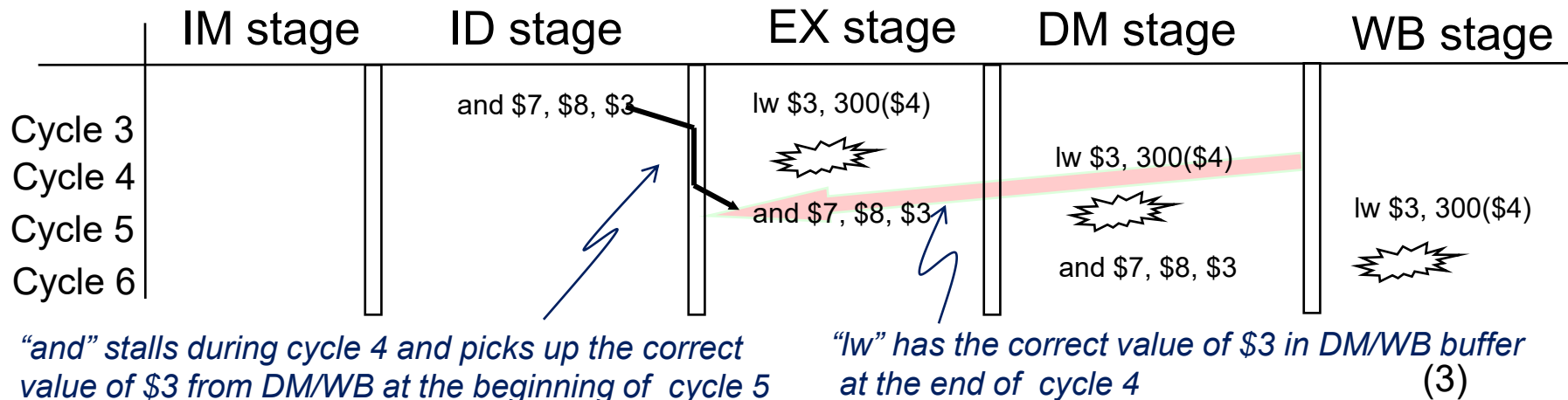


Forwarding may not be enough



Problem: can't use forwarding at the beginning of cycle 4 since “lw” produces the data to be written in \$3 at the end of cycle 4

Solution: need to combine forwarding with stalling the pipe.



Software Scheduling to Avoid Load Hazards

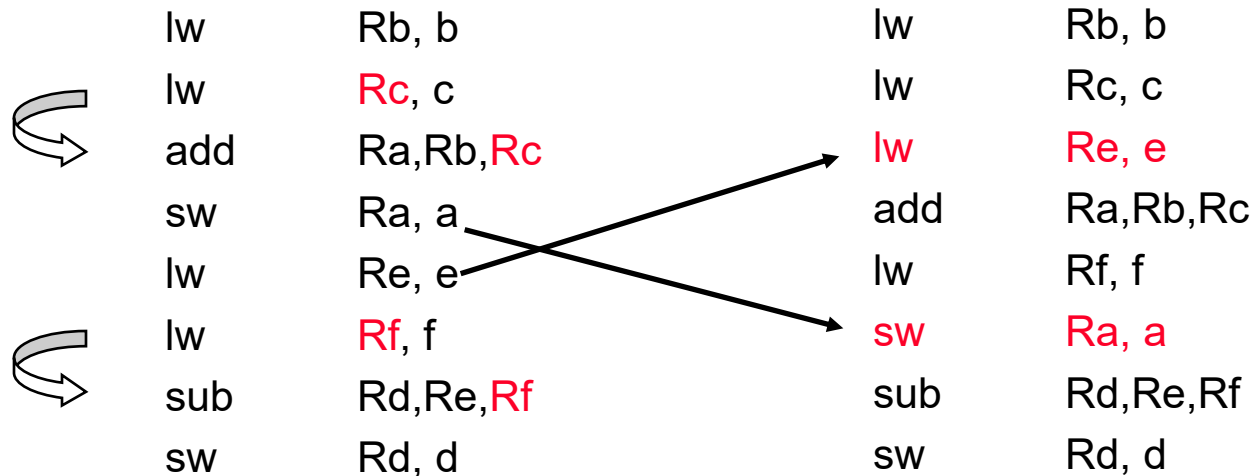
Try producing fast code for

$a = b + c;$

$d = e - f;$

assuming $a, b, c, d, e,$ and f in memory.

Slow code:



```
lw    Rb, b
lw    Rc, c
add   Ra, Rb, Rc
sw    Ra, a
lw    Re, e
lw    Rf, f
sub   Rd, Re, Rf
sw    Rd, d
```

Fast code:

```
lw    Rb, b
lw    Rc, c
lw    Re, e
add   Ra, Rb, Rc
lw    Rf, f
sw    Ra, a
sub   Rd, Re, Rf
sw    Rd, d
```

Review: Pipelining - What Makes it Hard ?



- Pipeline Hazards

- **structural hazards**: attempt to use the same resource by two different instructions at the same time
- **data hazards**: attempt to use data before it is ready
 - An instruction's source operand(s) are produced by a prior instruction still in the pipeline

- **control hazards**: attempt to make a decision about program control flow before the condition has been evaluated and the new PC target address calculated
 - branch and jump instructions, exceptions

- Pipeline hardware control must **detect** the hazard and then take action to **resolve** hazard

Control Hazards



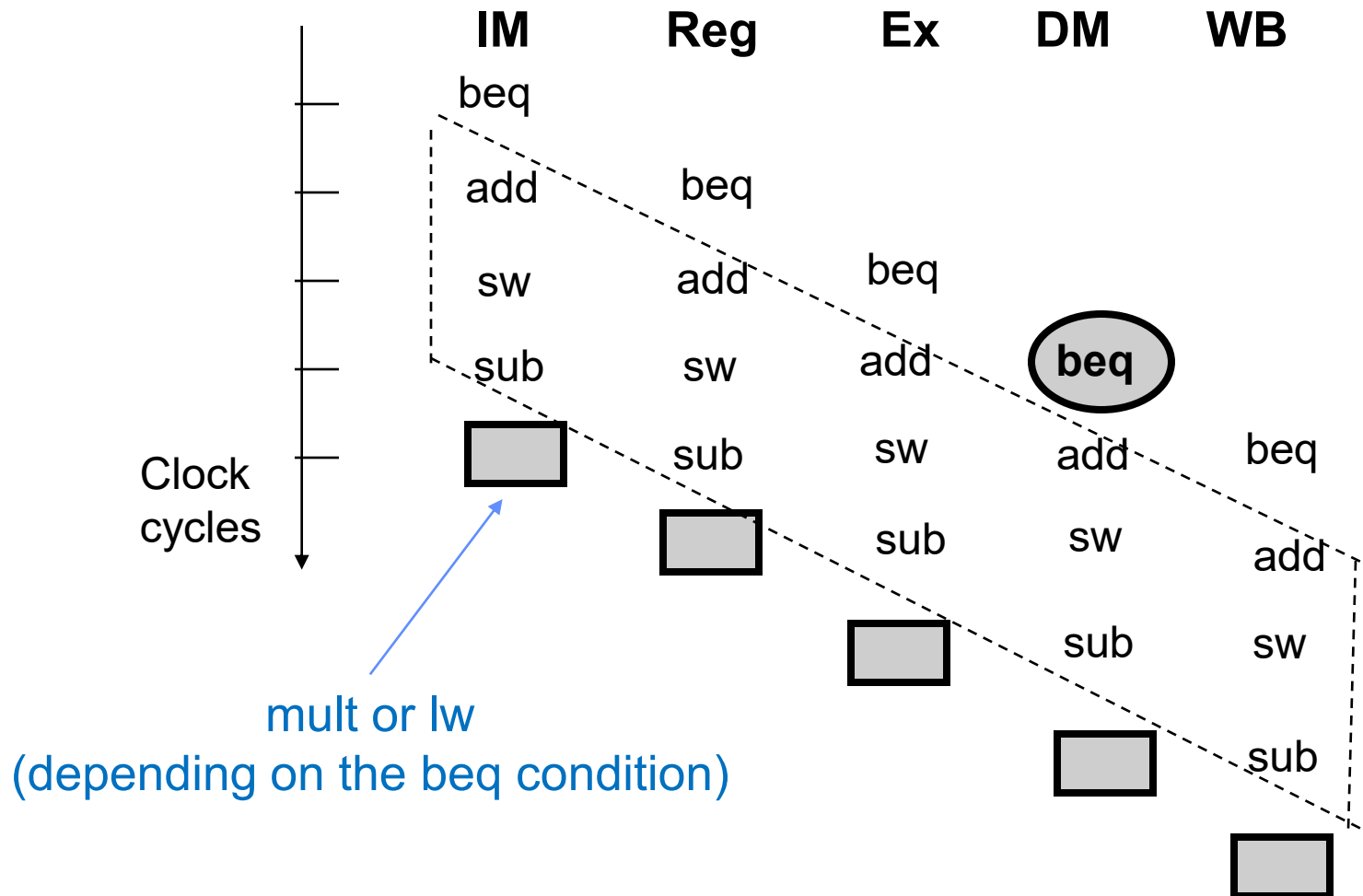
- When the flow of instruction addresses is not sequential (i.e., $PC = PC + 4$); incurred by change of flow instructions
 - Unconditional branches (`j`, `jal`, `jr`)
 - Conditional branches (`beq`, `bne`)
 - Exceptions
- Possible approaches
 - Stall (impacts CPI)
 - Move decision point as early in the pipeline as possible, thereby reducing the number of stall cycles
 - Delay decision (requires compiler support)
 - Predict and hope for the best !
- Control hazards occur less frequently than data hazards, but there is nothing as effective against control hazards as forwarding is for data hazards

Control (Branch, Jump) Hazards



- What do we need to know?
 - Next instruction target address, maybe sequential (PC+4)
or
 - » `beq`, PC+4 + branch instruction's sign-extended offset which is computed during EX by the Shift Left 2 – Add logic
 - » `j`, `jal`, constant address field read from IM during IF (26 bits)
 - » `jr`, `jalr`, read from RF during ID (32 bits)
 - » `trap` instruction or exception, obtained from table lookup in the OS (32 bits)
 - Branch decision outcome (ALU zero flag)
 - » continue sequentially? *or* jump to the branch target address?
- When do we need to know it?
 - As early as possible in the pipeline

Control Hazard on Branches



Control (Branch, Jump) Hazards, con't



- When do we act on the decision?
 - As soon as possible
 - » we decided too late? We already fetched another instruction, and may need to discard it (flush it)
 - Maybe will have to flush more than one instruction?
 - » we guessed, and were right – this is good
 - » we guessed, and were wrong – now need to fix things
 - Guesses require an evaluation of the success rate, by simulation (before the fact) or by measurement (after the fact)
 - » can measurement improve the quality of the guesses?
 - » recent history is often a reasonable predictor of the near future

Branch Stall Impact

- If $\text{CPI} = 1$, 10% branch, Stall 3 cycles $\Rightarrow$ new $\text{CPI} = 1.3$

For every 100 instructions:

- baseline cost = **100 cycles** (because $\text{CPI}=1$)
- number of branches = **10**
- extra stall cycles = **10 branches $\times$ 3 cycles = 30 cycles**

So total becomes **130 cycles for 100 instructions**, which means:

- new $\text{CPI} = 130 / 100 = 1.3$

Four Branch Hazard Alternatives



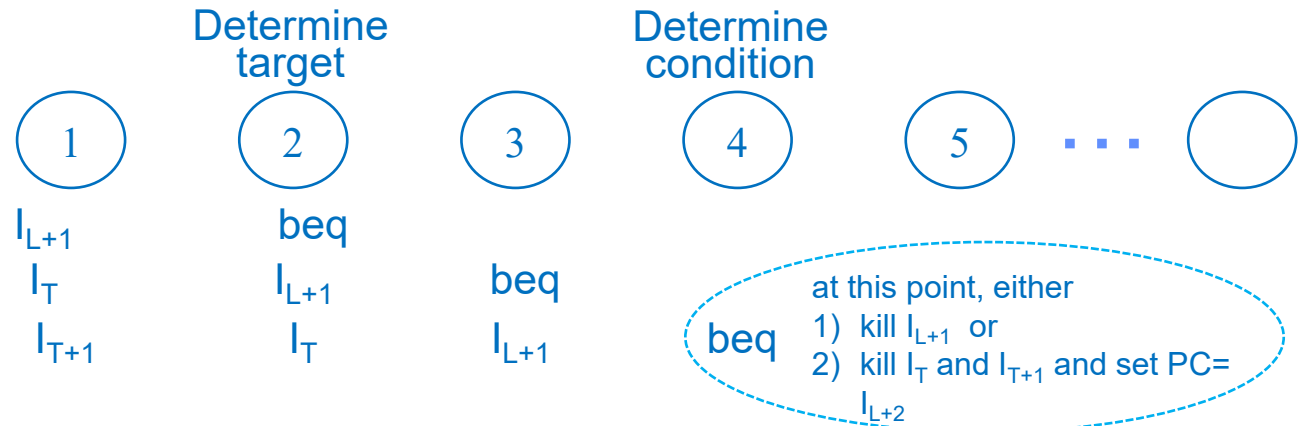
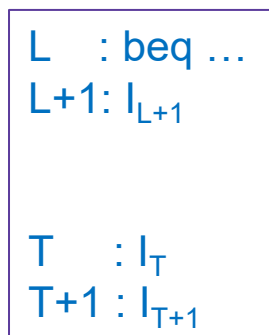
#1: Stall until branch direction is clear

#2: Predict Branch Not Taken

- Execute successor instructions in sequence (PC+4 already calculated)
- “Squash” instructions in pipeline if branch actually taken (how?)
- Advantage of late pipeline state update

#3: Predict Branch Taken

- More than 50% of MIPS branches are taken on average
- Start fetching from the branch target as soon as it is available
- Useful if target address is computed earlier than the branch condition (for example in stage 2 and stage 4, respectively, of a deep pipeline).



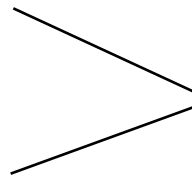
Four Branch Hazard Alternatives



#4: Delayed Branch

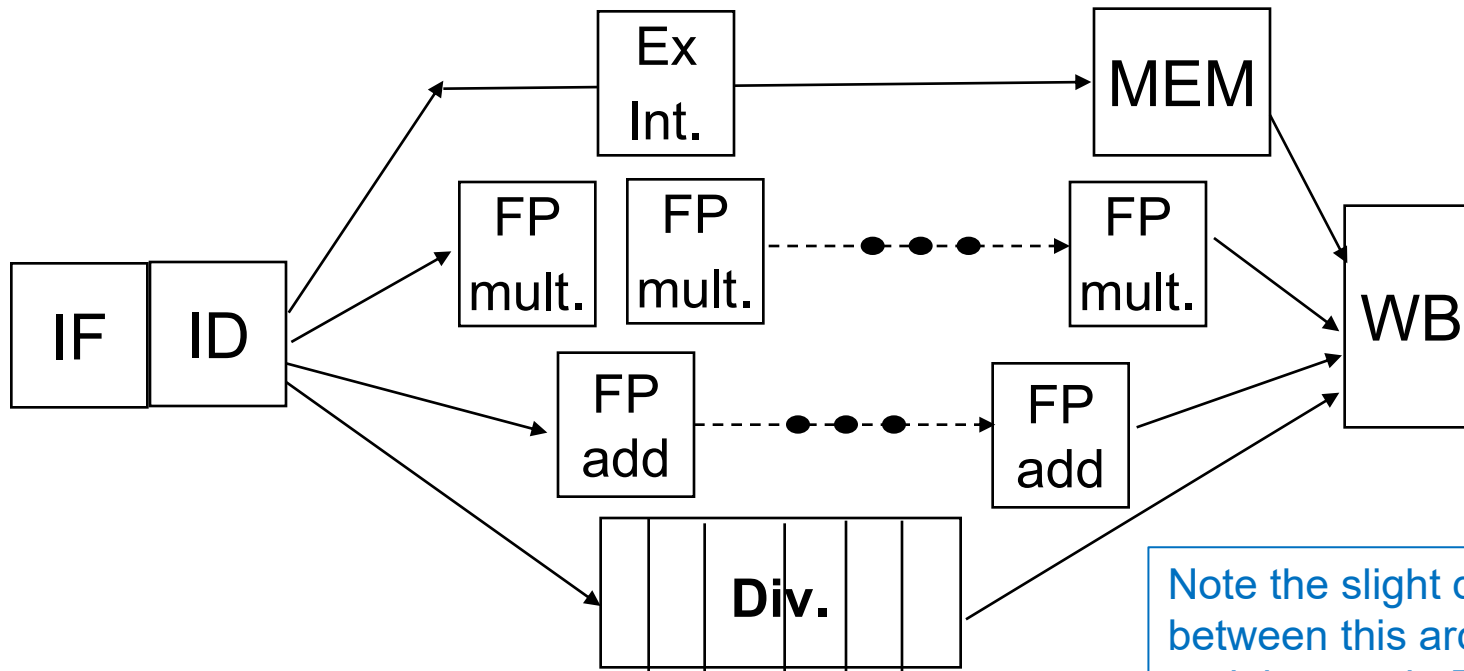
- Change semantics such that branching takes place **AFTER** the n instructions following the branch execute

```
branch instruction
  sequential successor1
  sequential successor2
  .....
  sequential successorn
branch target if taken
```

A diagram consisting of two lines that originate from a single point on the left and diverge to the right, forming a wide 'V' shape. This shape is used to represent a delay or a period of time.

Branch delay of length n

Multi-cycle pipelines (Section C.5)



Note the slight difference between this architecture and the one in Figure C.30 of the textbook

- **Assume**
 - 4-stage, pipelined, FP add
 - 7-stage, pipelined, FP multiply
 - 25 stage, non-pipelined divide unit

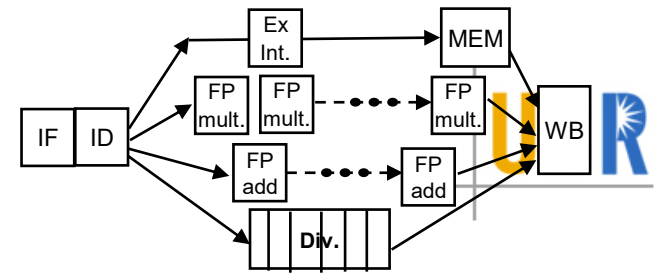
Hazards:

- Two divide instructions will stall the pipe (structural hazards).
- May have more than one register write in one cycle (why?)
 - increase number of ports, or stall the pipeline (interlock)
- May have WAW hazard (why?)
- Out-of-order completion causes problems with exceptions,
- The long pipes causes more RAW hazards (why?)

To deal with structural Hazards and resolve competition for the WB stage:

- Stall a conflicting instruction when entering the WB stage
 - may give priority to longer instructions to reduce RAW hazards.

Examples



Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
fld F4, 0(R2)	IF	ID	EX	M _{EM}	WB												
fmul.d F0, F4, F6		IF	ID	ID	M1	M2	M3	M4	M5	M6	M7	WB					
fadd.d F2, F0, F8			IF	IF	ID	ID	ID	ID	ID	ID	ID	A1	A2	A3	A4	WB	
fsd F2, 0(R2)					IF	IF	IF	IF	IF	IF	IF	ID	EX	EX	EX	M _{EM}	

Stall due to RAW hazards (assuming forwarding)

instruction	1	2	3	4	5	6	7	8	9	10	11
fmul.d F0,F4,F6	IF	ID	M1	M2	M3	M4	M5	M6	M7	WB	
...		IF	ID	EX	Mem	WB					
...			IF	ID	EX	Mem	WB				
fadd.d F2,F4,F6				IF	ID	A1	A2	A3	A4	WB	
...					IF	ID	EX	Mem	WB		
fld F2, 0(R2)						IF	ID	EX	Mem	WB	
							IF	ID	Ex	Mem	WB

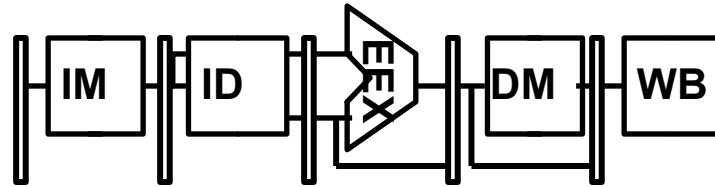
Structural hazards

(16)

Dealing with Exceptions

- Exceptions (aka interrupts) are just another form of control hazard. Exceptions arise from
 - R-type arithmetic overflow
 - Trying to execute an undefined instruction
 - An I/O device request
 - An OS service request (e.g., a page fault, TLB exception)
 - A hardware malfunction
- The pipeline has to stop executing the offending instruction in midstream, let all prior instructions complete, flush all following instructions, set a register to show the cause of the exception, save the address of the offending instruction, and then jump to a prearranged address (the address of the exception handler code)
- The software (OS) looks at the cause of the exception and “deals” with it

Where in the Pipeline Exceptions Occur



	Stage(s)?	Synchronous?
• Arithmetic overflow	EX	yes
• Undefined instruction	ID	yes
• TLB or page fault	IF, MEM	yes
• I/O service request	any	no
• Hardware malfunction	any	no

Be aware that multiple exceptions can occur simultaneously in a *single* clock cycle!